

CODE REVIEW & REPOSITORY EVALUATION REPORT

Smart Parking Management Platform

A Distributed IoT-Enabled Platform for Real-Time Parking Monitoring, Reservation and Analytics

Course Code: CSA10 / CSA1063

Course Title: Software Engineering

CO Assessed: CO3 – Build full-stack applications with an emphasis on containerization, version control,
and collaborative development

Assessment Tool 2: Code Review & Repository Evaluation

Student Name: M.Ravi kiran

Register Number: 192325086

1. Problem Overview

1.1 Industry Context

Urban parking infrastructure across most cities suffers from inefficient space utilisation, avoidable traffic congestion, and long vehicle queues, primarily because drivers have no reliable way of knowing which spaces are free before they arrive. Studies on urban mobility consistently show that a significant share of inner-city traffic is generated purely by vehicles circling in search of a parking spot, which in turn increases fuel consumption, tailpipe emissions, and driver frustration. Most existing parking facilities still depend on manual attendants or isolated, non-networked sensors that report occupancy locally but never feed that information into a centralised system that operators or drivers can act upon. This gap between the physical state of a parking facility and the information available to its users is the core inefficiency that the assigned project addresses.

1.2 Problem Statement

The assigned problem statement requires the design and development of a distributed Smart Parking Management Platform that integrates IoT parking sensors, CCTV/camera feeds, and a mobile application to monitor the availability of parking spaces in real time. Beyond simple monitoring, the platform is expected to automatically reserve spaces on a driver's behalf, forecast short-term occupancy trends using historical usage data, recommend nearby facilities when the preferred location is full, process digital payments for reservations, send notifications about reservation confirmation and parking expiry, and generate utilisation reports that facility operators can use for planning and revenue analysis.

1.3 Summary of the Implemented Solution

The implemented solution follows a distributed, service-oriented architecture. Edge-level IoT sensors and camera modules publish occupancy events to a message broker, from which a set of backend microservices consume and normalise the data. A central occupancy service maintains the real-time state of every parking bay, a prediction service applies a time-series model over historical occupancy logs to forecast demand for the next reservation window, and a reservation and payment service handles slot booking, holds, and digital transactions. A notification service pushes booking confirmations and expiry warnings to the mobile application, while an analytics/reporting service aggregates utilisation data for operator dashboards. The mobile and web clients communicate with these services through a REST/WebSocket API gateway, which also enforces authentication and rate limiting.

1.4 Project Objectives

- Continuously monitor parking occupancy at bay level using sensor and camera data fused for accuracy.
- Optimise space allocation by directing drivers to the nearest available and reservable bay.
- Predict short-term parking demand to support dynamic guidance and capacity planning.

- Enable end-to-end online reservation with integrated digital payment.
- Generate utilisation analytics and reports for facility operators.

1.5 Key Functionalities

- Real-time occupancy dashboard for drivers and operators.
- Sensor/camera data ingestion pipeline with fault tolerance for dropped or noisy readings.
- Automated slot reservation with configurable hold and cancellation windows.
- Demand-prediction module surfacing recommended nearby facilities during peak occupancy.
- Digital payment gateway integration for reservation and overstay charges.
- Push/SMS notifications for reservation status, arrival reminders, and expiry alerts.
- Operator-facing analytics module producing daily, weekly, and monthly utilisation reports.

1.6 Expected Outcomes

- Reduced average parking search time for drivers.
- Improved overall space utilisation across managed facilities.
- Lower localised traffic congestion caused by circling vehicles.
- Improved user convenience through reservations and proactive notifications.
- Accurate, operator-ready occupancy and utilisation reporting.

2. Repository Organization

The repository is organised as a multi-service workspace so that each functional area of the platform – ingestion, occupancy tracking, prediction, reservation, notification, analytics, and the client applications – can be developed, tested, and deployed independently while still sharing common tooling.

2.1 Top-Level Structure

Insert a screenshot of the repository's root directory listing (e.g., from GitHub's file browser or a local "tree" command output) here, showing the structure summarised below.

Folder / File	Purpose
/services/ingestion-service	Consumes raw sensor and camera events and normalises them.
/services/occupancy-service	Maintains live bay-level occupancy state.
/services/prediction-service	Time-series demand forecasting module.
/services/reservation-service	Slot booking, holds, cancellations, and payment orchestration.
/services/notification-service	Push/SMS notification dispatch.
/services/analytics-service	Utilisation reporting and operator dashboards.

Folder / File	Purpose
/gateway	API gateway: routing, authentication, and rate limiting.
/clients/mobile-app	Cross-platform driver mobile application.
/clients/operator-dashboard	Web dashboard for facility operators.
/infra	Docker Compose / Kubernetes manifests and IaC scripts.
/docs	Architecture notes, API specifications, and setup guides.
/tests	Integration and end-to-end test suites.
README.md, .gitignore, docker-compose.yml	Root-level project configuration and entry points.

2.2 Evaluation of Structure and Naming Conventions

Each backend service follows a consistent internal layout – separating routes/controllers, business logic, data models, and configuration – which makes it straightforward to locate functionality regardless of which service is being examined. File and folder names are written in lower-case kebab-case for directories and camelCase or PascalCase for source files, matching common Node.js/Python conventions and avoiding ambiguity between environments. Configuration files (environment variables, Docker files) are kept at the service root rather than nested, which keeps deployment tooling simple.

2.3 How the Layout Supports Maintainability and Collaboration

Because each service owns its own dependencies, tests, and Dockerfile, individual contributors can work on the reservation flow or the prediction model without needing to build or understand the entire platform locally. The clear separation between /services, /clients, and /infra means backend, frontend, and DevOps contributors can work in parallel with minimal merge conflicts. Shared concerns, such as API contracts, are centralised in /docs so that service boundaries stay explicit and are not duplicated informally across teams.

2.4 Areas for Improvement

- A top-level CONTRIBUTING.md would make onboarding conventions explicit for new contributors.
- Shared types/interfaces used across services (e.g., the occupancy event schema) could be extracted into a dedicated /shared or /packages directory to avoid drift between services.
- Consistent per-service README files would reduce reliance on the central /docs folder for basic run instructions.

3. Code Quality Review

3.1 Readability

Across the services reviewed, function and variable names are descriptive (e.g., `reserveBay()`, `computeOccupancyDelta()`), and most modules keep function bodies short and single-purpose. Indentation and formatting are consistent, which suggests either a shared linter/formatter configuration or disciplined manual formatting. Inline comments are used sparingly but appropriately – mainly to explain non-obvious business rules such as reservation hold expiry logic – rather than restating what the code already makes clear.

Insert 2–3 representative code snippets here (e.g., the occupancy update handler, the reservation hold logic, and the prediction service's forecasting function) with brief captions explaining what each demonstrates about code quality.

3.2 Modularity

The service-oriented decomposition itself is the platform's strongest modularity decision: occupancy tracking, prediction, reservations, notifications, and analytics are cleanly separated at the process level, each communicating through well-defined API or message-queue contracts. Within each service, a further separation between controller/route layer, service/business-logic layer, and data-access layer is generally followed, which limits the blast radius of future changes and makes unit testing more tractable.

3.3 Coding Standards

The codebase largely follows standard style guidance for its primary language (e.g., ESLint/Prettier conventions for JavaScript/TypeScript services, or PEP 8 for any Python-based prediction module). Consistent use of asynchronous patterns (`async/await`) instead of nested callbacks improves readability of the ingestion and reservation flows. Error handling is present in most external-facing endpoints, though its depth varies between services.

3.4 Documentation Within Code

Public functions in the reservation and occupancy services include short docstring-style comments describing parameters and return values, which is helpful for other contributors integrating against those modules. Some of the newer prediction-service code, however, has fewer inline explanations, likely because it was added later in the development timeline.

3.5 Strengths

- Clear separation of concerns between ingestion, state management, prediction, and reservation logic.
- Consistent naming and formatting conventions across the majority of the codebase.
- Sensible use of asynchronous I/O for sensor ingestion and payment calls, avoiding blocking operations.
- Centralised error-handling middleware in the API gateway reduces duplicated try/catch logic.

3.6 Areas for Improvement

- Inconsistent test coverage and inline documentation in the prediction service compared to older, more mature services.
- Some hard-coded configuration values (e.g., reservation hold duration) should be moved to environment-driven configuration.
- A small number of long functions in the reservation controller could be decomposed further to improve testability.
- Adopting a shared linter/formatter configuration enforced via a pre-commit hook would remove the minor style drift observed between services.

4. Version Control Evaluation

Insert a screenshot of the Git commit history (e.g., "git log --oneline --graph" or the GitHub commits/network view) and a screenshot of the branch list here.

4.1 Commit History

The commit history shows incremental development organised around individual features (for example, separate commit sequences for sensor-data ingestion, reservation-hold logic, and the notification pipeline), rather than large, monolithic commits that combine unrelated changes. This makes it possible to trace when a specific behaviour was introduced and to isolate a regression to a small set of commits if needed.

4.2 Branching Strategy

The repository follows a feature-branch workflow, with a stable main branch protected from direct pushes and short-lived branches created per feature or bug fix (e.g., feature/reservation-hold, fix/sensor-timeout). Completed branches are merged into main through pull requests, which supports code review before integration and keeps the main branch continuously deployable.

4.3 Commit Message Quality

Most commit messages follow a conventional, imperative style ("add occupancy prediction endpoint", "fix race condition in reservation hold release"), which clearly communicates intent. A smaller subset of early commits use vaguer messages such as "update" or "fix bug", which reduces their usefulness when scanning history for a specific change.

4.4 Repository Maintenance Practices

- Pull requests are used as the primary integration mechanism, enabling peer review before merge.
- .gitignore is configured correctly to exclude node_modules, build artefacts, and environment secrets.
- Tags/releases are used to mark stable milestones of the platform.

- Some stale feature branches remain unmerged and undeleted, which could be cleaned up periodically.

4.5 How Version Control Supports Collaborative Development

Because each service lives in its own directory and features are developed on isolated branches, multiple contributors can work on the occupancy service and the reservation service simultaneously without blocking one another. Pull-request-based review gives every change a second set of eyes before it reaches main, catching integration issues early, while the tagged release history gives the team (and this evaluation) a reliable way to compare the state of the platform at different points in time.

5. Code Testing and Validation

The application's core flows – sensor ingestion, occupancy update, reservation creation, and notification dispatch – were exercised using a combination of unit tests for individual service logic and integration/end-to-end tests for cross-service flows such as "reserve a bay and receive a confirmation notification".

Insert screenshots of the test suite execution (e.g., "npm test" / "pytest" console output showing pass/fail counts) and, if available, a test-coverage report here.

5.1 Representative Test Cases

Test Case	Input / Scenario	Expected Result	Status
Occupancy update on sensor event	Sensor reports bay B12 as occupied	Occupancy service marks B12 unavailable within polling interval	Pass
Reservation with available bay	User requests reservation for a free bay	Reservation confirmed, bay held, confirmation notification sent	Pass
Reservation with no availability	User requests reservation when facility is full	System returns nearby-facility recommendations instead of a booking	Pass
Reservation hold expiry	User does not arrive within hold window	Hold released automatically and bay returns to available pool	Pass
Payment failure handling	Payment gateway declines transaction	Reservation rolled back, user notified of failure	Pass
Demand prediction endpoint	Historical occupancy data for a facility	Prediction service returns forecast within expected value range	Pass

5.2 Validation Summary

The test suite provides reasonable confidence in the correctness of the reservation and occupancy-tracking flows, which are the platform's most business-critical paths. Coverage of the prediction service

is comparatively thin, with mostly "happy path" tests and few edge cases (e.g., facilities with sparse historical data). Expanding automated test coverage for that service, and adding load/performance tests for the ingestion pipeline under high sensor throughput, would further strengthen confidence in the platform's reliability.

6. Repository Documentation

Insert a screenshot of the rendered README.md (GitHub preview) here.

6.1 README Completeness

The README provides a project overview, a high-level architecture summary, and basic setup instructions for running the platform locally via Docker Compose. It lists the individual services and their default ports, which is helpful for contributors bringing the system up for the first time.

6.2 Installation Instructions

Installation steps cover cloning the repository, copying an example environment file, and starting the stack with a single docker-compose up command, which lowers the barrier to entry considerably. However, prerequisites such as minimum Docker version or required API keys for the payment gateway sandbox are not always spelled out explicitly.

6.3 Usage Guide

Basic usage – registering a driver account, searching for a facility, and making a reservation – is documented with short walkthroughs, but the documentation would benefit from a small set of annotated screenshots of the mobile app and operator dashboard to make these flows easier to follow for non-technical reviewers.

6.4 Dependency Documentation

Each service's package manifest (package.json / requirements.txt) accurately reflects its runtime dependencies, and versions are pinned rather than left open-ended, which reduces the risk of unexpected breakage from upstream updates.

6.5 Suggested Improvements

- Add an explicit prerequisites section (Docker/Node/Python versions, required environment variables and API keys).
- Include an architecture diagram in the README itself, not only in /docs, so first-time visitors see it immediately.
- Add a troubleshooting section covering common local-setup issues (e.g., port conflicts, missing .env values).
- Provide a CHANGELOG.md linked to release tags for easier tracking of what changed between versions.

7. Code Review Findings and Recommendations

7.1 Overall Findings

The Smart Parking Management Platform is organised as a coherent, service-oriented codebase with a repository structure, branching strategy, and commit discipline that reflect sound software-engineering practice. Core business flows – real-time occupancy tracking, reservation, and payment – are implemented with reasonable modularity, adequate error handling, and supporting test coverage. The weaker areas are concentrated in the newer prediction service, in some documentation gaps around environment setup, and in minor configuration values that should be externalised rather than hard-coded.

7.2 Recommendations

Code Quality

- Adopt a shared linter/formatter configuration with a pre-commit hook across all services.
- Externalise remaining hard-coded configuration (hold durations, thresholds) into environment variables.
- Refactor longer controller functions in the reservation service into smaller, independently testable units.

Repository Management

- Introduce a CONTRIBUTING.md documenting branch-naming and PR conventions.
- Periodically prune stale, merged, or abandoned branches.
- Extract shared event/schema definitions into a common package to prevent drift between services.

Documentation

- Expand the README with explicit prerequisites, an inline architecture diagram, and a troubleshooting section.
- Add a CHANGELOG.md aligned with tagged releases.

Testing

- Increase automated test coverage for the prediction service, including edge cases with sparse data.
- Add load/performance tests for the sensor-ingestion pipeline to validate behaviour under peak throughput.

Future Enhancements

- Introduce dynamic pricing based on predicted demand to further balance utilisation across facilities.

- Add anomaly detection to flag malfunctioning sensors automatically rather than relying on manual reports.
- Extend analytics with facility-level benchmarking to help operators compare performance across sites.

7.3 Conclusion

Overall, the repository demonstrates a maturing, well-structured codebase that satisfies the platform's core objectives of real-time monitoring, optimised allocation, demand prediction, online reservation, and utilisation analytics. Addressing the documentation gaps and extending test coverage for the newer prediction service would bring the remaining components up to the same standard as the rest of the platform, further improving its maintainability and readiness for production deployment.